Детский технопарк «Альтаир»

(РТУ МИРЭА)

«ОСНОВЫ ПРОМЫШЛЕННОГО ПРОГРАММИРОВАНИЯ»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

«GoldForum — платформа для ведения блогов с векторным поиском»

Верниковский Виталий Викторович

Асонов Ефим Андреевич

Ученики группы 2

Руководитель: Борисов Артём Игоревич

Преподаватель Детского технопарка «Альтаир» РТУ МИРЭА

Москва, 2026

Название проекта

GoldForum — платформа для ведения блогов с векторным поиском

Авторы проекта

Верниковский Виталий Викторович и Асонов Ефим Андреевич, гр. 2

Описание идеи

GoldForum — это полноценная веб-платформа для ведения блогов, объединяющая классическую функциональность форума с современными технологиями машинного обучения. Проект создан для того, чтобы продемонстрировать применение промышленных подходов к разработке веб-приложений на языке Python: от проектирования базы данных и REST API до внедрения рекомендательной системы на основе семантических векторных представлений текста.

Основные возможности:

- Регистрация и авторизация пользователей с безопасным хешированием паролей.
- Создание, редактирование и удаление постов с Markdown-контентом.
- Загрузка изображений и файлов к постам.
- Система комментариев, лайков, избранного и отметок «прочитано».
- Профиль пользователя с четырьмя вкладками (Мои посты / Понравившиеся / Избранное / Прочитанные).
- Полнотекстовый векторный (семантический) поиск по постам и авторам.
- Бесконечная лента рекомендаций (infinite scroll) на основе ML-алгоритма с FIFO-буферами взаимодействий.
- Импорт и экспорт постов в ZIP-архивах.
- Полноценный REST API (две версии: v1 на Flask Blueprints и v2 на Flask-RESTful).

Актуальность: в условиях роста объёмов пользовательского контента возникает потребность в интеллектуальных механизмах фильтрации и рекомендаций. GoldForum решает эту задачу путём анализа семантической близости постов с использованием нейросетевых эмбеддингов, что позволяет рекомендовать пользователю релевантный контент даже при отсутствии явных ключевых слов.

Описание реализации

Архитектура приложения

Проект построен по модульной архитектуре с разделением на слои. Точкой входа является файл `app.py`, в котором создаётся и конфигурируется экземпляр Flask-приложения. В функции `main()` последовательно выполняются: регистрация 32 кастомных обработчиков HTTP-ошибок (от 400 до 505, включая шуточный 418 «I'm a teapot»), инициализация базы данных SQLite через SQLAlchemy, создание пользователя по умолчанию (`admin`/`admin`), синхронизация всех существующих постов с векторной базой ChromaDB, регистрация восьми API-Blueprint'ов и двух Flask-RESTful-ресурсов.

Слой базы данных включает четыре модели SQLAlchemy: `UserModel` (пользователи с поддержкой Flask-Login UserMixin), `PostModel` (посты с Markdown-контентом и метаданными загруженных файлов), `CommentModel` (комментарии с древовидной структурой через внешние ключи) и `UserPostInteraction` (флаги взаимодействий: лайк, избранное, прочитано, не интересно). Для сериализации моделей используется библиотека `SQLAlchemy-serializer`, предоставляющая метод `to_dict()`. Пароль пользователя хранится в виде хеша, вычисленного функцией `werkzeug.security.generate_password_hash()`. Все связи между таблицами настроены с каскадным удалением (при удалении пользователя удаляются его посты, при удалении поста — комментарии и взаимодействия).

Рекомендательная система и векторный поиск

Ключевой особенностью проекта является рекомендательная система, реализованная в модуле `backend/recommendation/__init__.py`. Алгоритм использует два независимых FIFO-буфера (Alpha и Beta) для каждого пользователя. При каждом взаимодействии (лайк, избранное, прочитано) в оба буфера добавляется пара: семантический вектор поста (получаемый из ChromaDB) и весовые очки (1 — прочитано, 3 — лайк, 5 — избранное). Размеры буферов настраиваются в `config.json` (Alpha — 25 элементов, Beta — 10). Вектор профиля пользователя вычисляется как взвешенное усреднение векторов в буфере: `SUM(vector_i × score_i) / SUM(score_i)`. Для предотвращения создания гигантских промежуточных массивов применяется математически эквивалентная оптимизация без физического расширения данных.

Целевой вектор для поиска кандидатов выбирается вероятностно: с вероятностью 50% используется вектор Alpha, с 30% — Beta, с 20% — случайный пост (с вероятностью выбора, пропорциональной `√(likes_count)`). Кандидатами считаются посты, косинусное расстояние до целевого вектора которых меньше порога TBM = 0.3. Из кандидатов один пост выбирается с вероятностью, пропорциональной `√(likes_count + 1)`. При отсутствии кандидатов срабатывает fallback — случайный пост из всей базы. Батч рекомендаций генерируется без дубликатов (с учётом `exclude_ids` — уже показанных постов) и отдаётся клиенту через API `/api/feed` для реализации infinite scroll.

Векторная база данных построена на связке **ChromaDB** (PersistentClient, коллекция «posts» с метрикой cosine) и SentenceTransformers (модель `paraphrase-multilingual-MiniLM-L12-v2`, поддерживающая русский и английский языки). Класс `VectorDB` реализован как потокобезопасный синглтон (паттерн `__new__`). При создании поста его текст (заголовок +

содержимое) векторизуется и сохраняется в ChromaDB. При каждом запуске сервера выполняется полная синхронизация: все посты из SQLite сверяются с векторной базой и при необходимости добавляются или обновляются. Модуль также предоставляет метод `search_posts()` для семантического поиска, возвращающий результаты, ранжированные по косинусному расстоянию.

Описание технологий

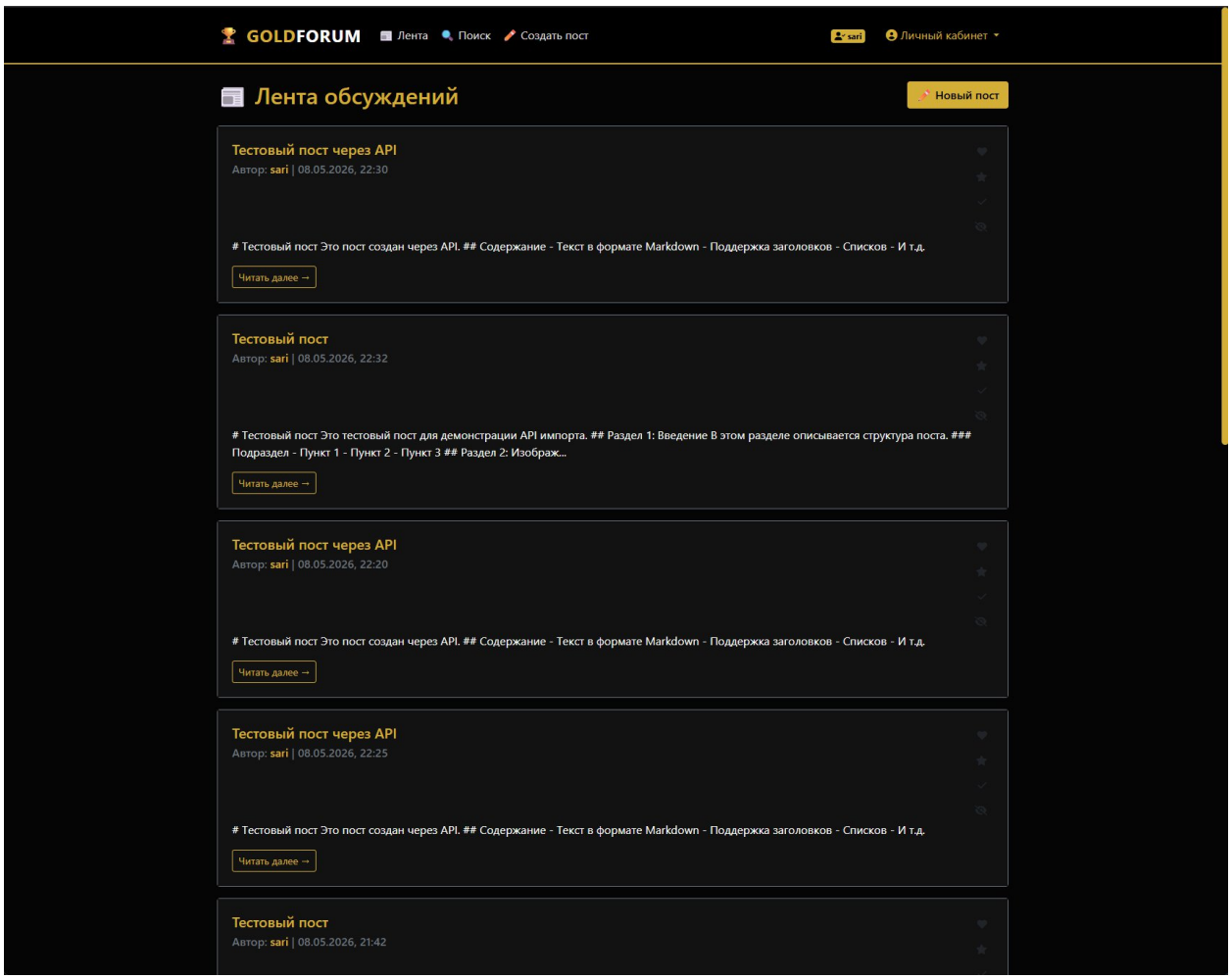
Бэкенд: Python 3.10+ с микро-веб-фреймворком Flask (маршрутизация, шаблонизатор Jinja2). Для работы с базой данных SQLite используется ORM SQLAlchemy (четыре модели: пользователи, посты, комментарии, взаимодействия). Аутентификация и сессии реализованы через Flask-Login. Генерация и валидация форм — Flask-WTF/WTForms с защитой от CSRF. REST API v2 реализован на Flask-RESTful в объектно-ориентированном стиле.

ML и векторный поиск: семантические эмбеддинги текста создаются моделью `paraphrase-multilingual-MiniLM-L12-v2` (SentenceTransformers, ~470 МБ, поддержка русского и английского языков). Векторная база — ChromaDB (PersistentClient) с индексом HNSW и метрикой cosine. Векторные операции (сложение, норма, косинусное расстояние) — NumPy.

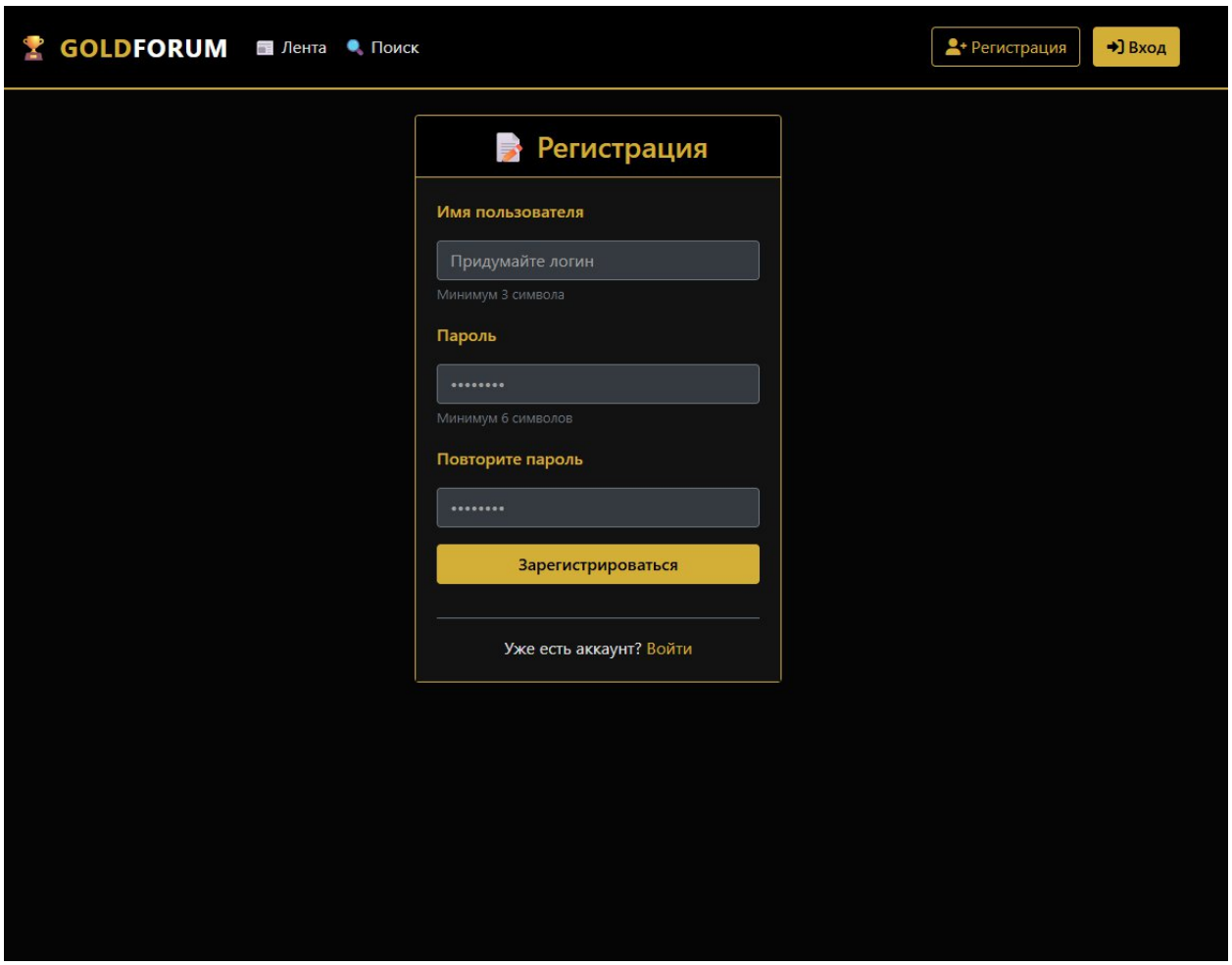
Фронтенд: Bootstrap 4 (адаптивная вёрстка), Font Awesome 5 (иконки), marked.js (клиентский рендеринг Markdown для предпросмотра).

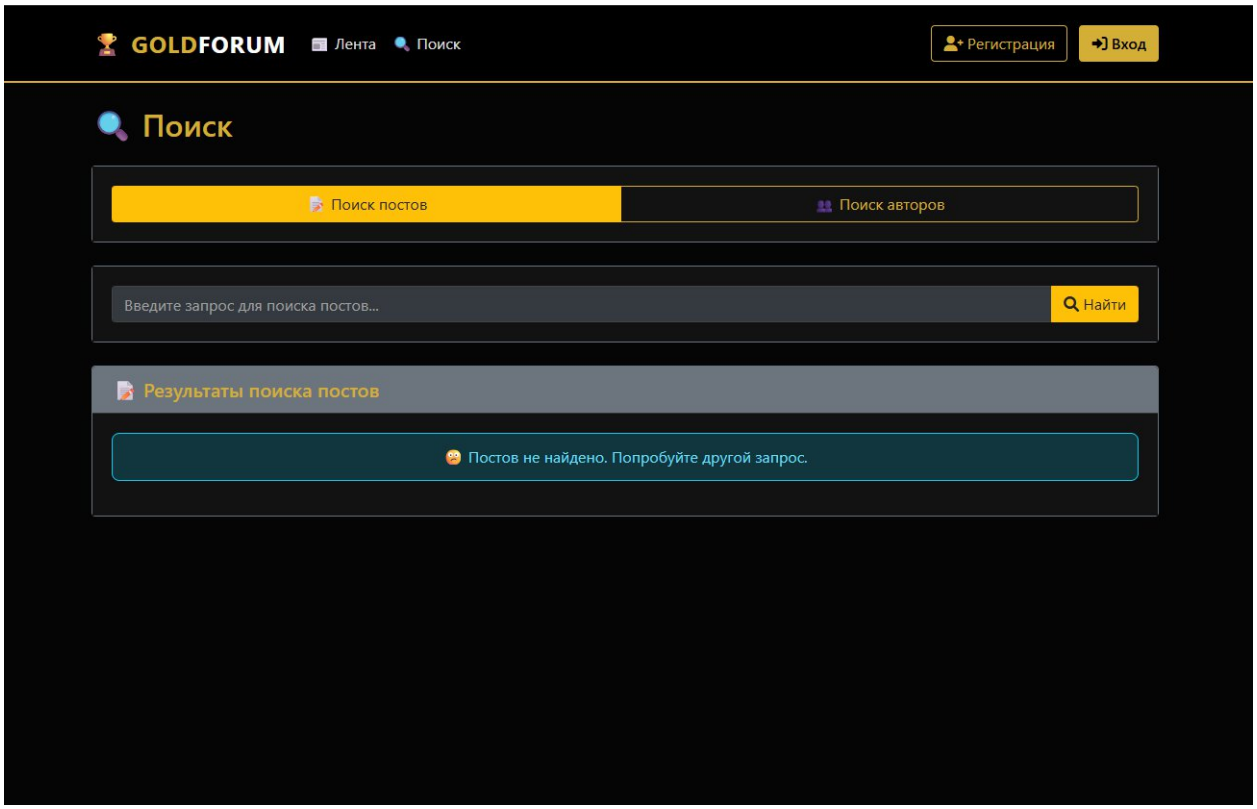
Инструменты: Git, виртуальное окружение venv, Alembic (миграции БД).

Результат работы

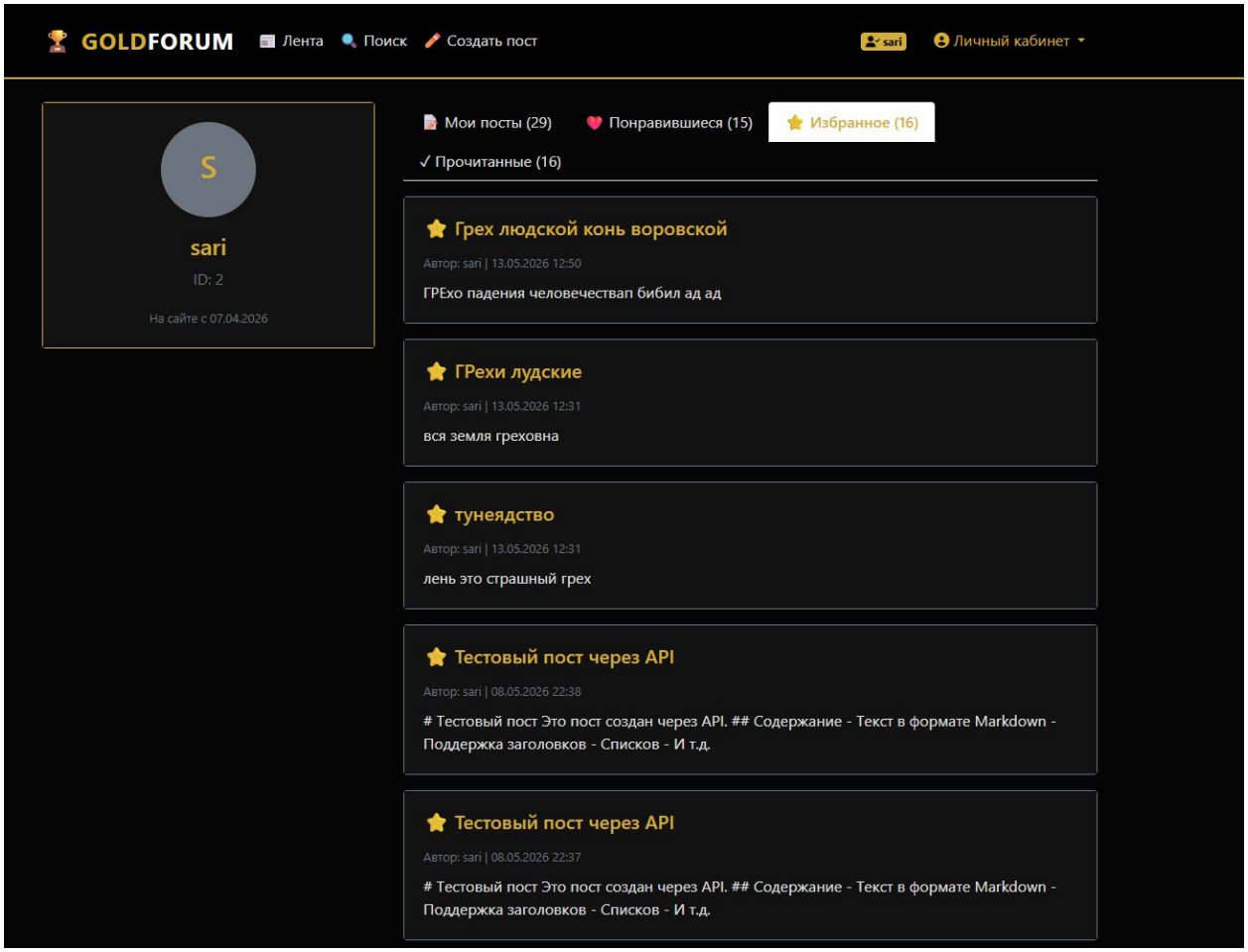


Главная страница (`/`) отображает ленту постов. Для неавторизованных пользователей показываются последние 50 постов в хронологическом порядке. Для авторизованных пользователей работает бесконечная лента рекомендаций (infinite scroll): при прокрутке страницы вниз автоматически подгружается следующий батч из 10 постов, сгенерированный рекомендательной системой. У каждого поста в ленте отображаются: заголовок, автор, первые 200 символов содержимого, количество лайков, дата создания и четыре кнопки взаимодействий (лайк, избранное, прочитано, скрыть).


GOLDFORUM
Лента
Поиск
Регистрация
Вход
Регистрация
Имя пользователя
Придумайте логин
Минимум 3 символа
Пароль
Минимум 6 символов
Повторите пароль
Зарегистрироваться
Уже есть аккаунт? Войти


GOLDFORUM
Лента
Поиск
Регистрация
Вход
Поиск
Поиск постов
Поиск авторов
Введите запрос для поиска постов...
Найти
Результаты поиска постов
Постов не найдено. Попробуйте другой запрос.

Страница поиска (`/search?q=...`) поддерживает два режима: векторный (семантический) поиск по содержимому постов с ранжированием по релевантности и нестрогий текстовый поиск по авторам (SQL `LIKE '%query%'`). Результаты выводятся в двух отдельных блоках.



Страница профиля (`/profile/<user_id>`) для владельца аккаунта отображает четыре вкладки: «Мои посты», «Понравившиеся», «Избранное», «Прочитанные». Для чужого профиля — только список постов автора. Каждая вкладка загружает данные динамически через API.

Руководство запуска

Установка и запуск

1. **Клонирование репозитория:**
2. **Создание виртуального окружения:**
3. **Активация виртуального окружения:**
4. **Установка зависимостей:**

Полный список зависимостей: `flask`, `flask-wtf`, `sqlalchemy`, `flask-login`, `SQLAlchemy-serializer`, `flask-restful`, `alembic`, `markdown`, `werkzeug`, `chromadb`, `sentence-transformers`. Установка может занять 5–10 минут, так как `sentence-transformers` скачивает ML-модель (~470 МБ) и зависимости PyTorch.

5. **Запуск сервера:**

6. **Открыть в браузере:** http://127.0.0.1:5000

Первый запуск

При первом запуске автоматически выполняются следующие действия:

- Создаётся файл базы данных SQLite `data/sql_db/forum.db` со всеми необходимыми таблицами.

- Создаётся пользователь по умолчанию с логином `admin` и паролем `admin`.

- Инициализируется векторная база ChromaDB в директории `data/vector_db/`.

- Загружается и кешируется ML-модель эмбеддингов (при последующих запусках используется кешированная версия).

Для сброса базы данных и начала с чистого листа достаточно удалить папку `data/sql_db/` и перезапустить сервер.

Руководство пользователя

Регистрация и вход

1. Перейдите на страницу /register для создания нового аккаунта. Введите желаемые логин и пароль (минимум 6 символов).

2. После успешной регистрации вы будете автоматически перенаправлены на страницу входа /login.

3. Войдите под своей учётной записью. Сессия сохраняется на 30 дней.

Создание и публикация поста

1. Нажмите кнопку «Новый пост» в навигационной панели или перейдите по адресу /post/new.

2. Заполните заголовок поста.

3. В поле содержимого используйте Markdown-разметку. Доступные элементы:

- `# Заголовок 1`, `## Заголовок 2`, `### Заголовок 3`
- `**жирный текст**`, `*курсив*`
- `- элемент списка`, `1. нумерованный список`
- `[текст ссылки](https://...)`
- `![альт текст](url-изображения)`
- `` `код` `` или блоки кода через ```` ```

4. Предпросмотр рендеринга отображается справа в реальном времени.

5. При необходимости загрузите изображения (png, jpg, gif, webp, bmp, svg — до 16 МБ) и файлы (pdf, doc, txt, zip и др.) с помощью кнопок загрузки.

6. Нажмите «Опубликовать». После создания пост появится в ленте и будет проиндексирован в векторной базе.

Взаимодействие с постами

На главной странице и странице поста доступны четыре кнопки взаимодействий

Все взаимодействия отправляются через AJAX без перезагрузки страницы и сразу же влияют на рекомендательную ленту.

Бесконечная лента рекомендаций

После авторизации главная страница переключается в режим рекомендаций:

- При входе на страницу загружается первый батч из 10 рекомендованных постов.

- Когда вы прокручиваете страницу до конца, автоматически загружается следующий батч (infinite scroll).

- Рекомендации формируются на основе ваших взаимодействий: чем больше вы ставите лайков и добавляете в избранное, тем точнее система подбирает похожий контент.

- Уже показанные посты не повторяются в рамках одной сессии.

Профиль пользователя

Перейдите по адресу /profile/<user_id> (или кликните на имя автора любого поста):

- Владелец профиля видит четыре вкладки: «Мои посты», «Понравившиеся», «Избранное», «Прочитанные». Вкладки загружаются динамически.

- Гость / другой пользователь видит только список постов автора.

Поиск

1. Введите запрос в строку поиска в навигационной панели.

2. Результаты поиска (/search?q=ваш_запрос) отображаются в двух секциях:

  - Посты: семантический поиск с ранжированием по релевантности. Находит посты, похожие по смыслу, а не только по точному совпадению слов.

  - Авторы: текстовый поиск по именам пользователей (частичное совпадение).

Комментарии

На странице поста под содержимым расположена секция комментариев:

- Просматривайте комментарии других пользователей с указанием автора и даты.

- Добавляйте свои комментарии через форму ввода. Поддерживается Markdown-разметка.

- Удаляйте собственные комментарии с помощью кнопки «Удалить».

При возникновении ошибок отображается кастомизированная страница с иконкой, кодом ошибки, заголовком и описанием на русском языке. Поддерживаются все стандартные коды HTTP-ошибок от 400 до 505.